

Business Harness Bench

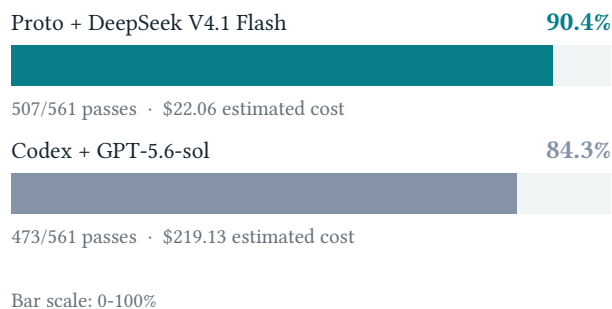
Evaluating Agents on Business Deliverables

ERP AI

Code, tasks and evaluation evidence

Abstract. Business agents are useful when the work they deliver is correct, usable, and repeatable. We introduce Business Harness Bench, a runnable benchmark comprising 187 file-based desk tasks across seven categories and 20 application-building tasks with three successive change requests. Desk tasks combine heterogeneous inputs, explicit business rules, reference solutions, and executable checks; build tasks evaluate delivery, permissions, data integrity, and behavior under change. We report a complete desk comparison of Proto with DeepSeek V4.1 Flash and Codex with GPT-5.6-sol, with three attempts per task and a shared frozen scorer. Proto passes 507 of 561 attempts (90.4%), compared with 473 of 561 (84.3%) for Codex, a difference of 6.06 percentage points. Captured-usage cost estimates are \$22.06 and \$219.13, respectively. These are results for two configured agent systems on a development task set, not isolated model or harness effects. The release includes task packs, runners, the frozen scorer, and an attempt-level evidence ledger; application tasks are released without a completed build leaderboard.

A Overall task success



B Success by repetition

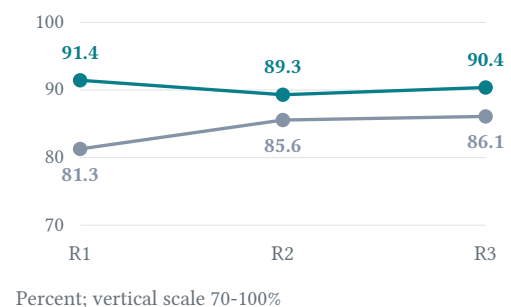


Figure 1. Full desk comparison under the shared frozen scorer. **A:** aggregate success over 561 attempts per system, with captured-usage cost estimates. **B:** success in each complete 187-task repetition. Teal: Proto + DeepSeek; slate: Codex + Sol. Costs are estimates, not invoices.

1. Introduction

A business owner delegates a reconciliation because a payment decision depends on it, not because a description of reconciliation is needed. The same distinction applies to an import file, a monthly report, or an internal application: the agent's response is not the deliverable. The relevant question is whether the owner can use what the agent hands back without repairing the work.

Business correctness is often conjunctive. A workbook may show the right total while its formulas fail. An import may be syntactically valid while dropping customers. A payment may match an amount but refer to the wrong invoice. In an application, hiding a button does not enforce authorization. These failures motivate evaluation against explicit delivery contracts rather than fluency, self-reported completion, or activity counts.

2. Benchmark design

Business Harness Bench tests two forms of delegation. In the **desk track**, an agent receives a folder and a short request, then produces files. In the **build track**, an agent receives a business requirement and seed data, delivers an application, and applies three changes. Both tracks evaluate the handoff; their scores remain separate because file checks and application acceptance are not interchangeable units.

2.1 Desk tasks

The desk track contains 187 tasks. Inputs include CSV and XLSX exports, text and scanned documents, contextual messages, and occasional databases. Tasks require agents to reconcile inconsistent records, apply supplied policies, preserve required fields, and distinguish supported conclusions from missing evidence. Fixtures are generated business-shaped data, not private customer engagements or a statistically representative sample of business demand.

Category	Tasks	Representative output
Spreadsheet	41	Reconciled or calculated workbook
Bookkeeping	36	Reconciliation, schedule, classification
Reports	30	Data summary and supported memo
Reformatting	26	Destination-compatible import file
Extraction	26	Structured records from documents
Drafting	19	Business text preserving supplied facts
Tooling	9	Small file-based tool or static page
Total	187	Three attempts per reported system

Table 1. Composition of the desk track. Each task supplies an ask, workspace, generator, checks, and reference solution. Checks and references remain outside the evaluated agent's container.

2.2 Application tasks

The 20 build tasks cover CRM, inventory, orders, field service, leave, purchase approvals, time tracking, memberships, events, assets, helpdesk, vendors, expenses, quotes, donors, appointments, property maintenance, recruiting, work orders, and point of sale. A shared enterprise baseline addresses delivery, invitations, roles, server-side scope, live dashboards, exact lists and exports, audit metadata, validation, and persistence. Domain-specific checks add the business rules.

Each application receives three successive change requests. The runner carries the prior workspace forward, but invokes a new one-shot turn rather than claiming conversational continuity. Testers recheck existing requirements after each change. Automated probes support delivery checks and produce a tester sheet; they do not establish working permissions or complete acceptance. No completed build leaderboard is reported here.

2.3 Contribution and scope

The contribution is an inspectable evaluation package joining portable artifact contracts with application handoff under change. It complements enterprise interaction settings such as WorkArena [1], spreadsheet manipulation in SpreadsheetBench [2], and simulated professional work in TheAgentCompany [3]. EnterpriseClawBench [4] evaluates workspace tasks recovered from real workplace sessions. Our inputs are authored fixtures; we do not claim the same source provenance. The benchmark measures configured systems, including models, tools, skills, and execution substrate.

3. Evaluation protocol

3.1 Workload and configurations

The reported comparison covers all 187 desk tasks three times for each system: **561 attempts per system and 1,122 attempts in total**. Every repetition is retained. The Proto cohort uses runtime revision c8f62dd60 with DeepSeek V4.1 Flash, high reasoning, temperature 0, and DeepSeek-only routing without fallback. Codex uses CLI 0.154.0, GPT-5.6-sol, and high reasoning. Codex sampling is CLI-managed and is not asserted to match Proto's temperature.

The Proto cohort comprises one full repetition and two additional full repetitions of the same runtime. Logical repetition identifiers are preserved in the released ledger. Codex's three full repetitions are reused from its completed benchmark cohort. These are matched task sets under a shared scorer, not simultaneous randomized trials or a same-model harness ablation. Later Proto-versus-Proto development comparisons are outside this release.

3.2 Frozen artifact scoring

All attempts are scored with the same **conservative-v7** package. Each receipt binds the verdict to an original result hash, output-artifact hashes, and the scorer manifest. The release includes the immutable scorer source and task definitions. Export verifies each original result hash and receipt identity; local verification checks every file named by the frozen manifest.

Required checks are conjunctive: a task passes only when every required check passes. Workbook scoring uses native recalculation after cached values are stripped. The frozen scorer includes conservative equivalence checks for alternative valid representations. Original raw verdicts remain separately available; they are not mixed with frozen verdicts in the primary score. There are four original grader-error attempts for Proto and three for Codex, and zero errors under the frozen scorer.

3.3 Results

Measure	Proto + DeepSeek V4.1 Flash	Codex + GPT-5.6-sol
Repetition 1	171/187	152/187
Repetition 2	167/187	160/187
Repetition 3	169/187	161/187
Frozen score	507/561 (90.4%)	473/561 (84.3%)
All three attempts pass	147/187	141/187
Original raw score	447/561 (79.7%)	431/561 (76.8%)

Table 2. Complete desk comparison. The primary score is the frozen artifact verdict. Every passing artifact also completed normally; no passing timeout inflates the normal-completion count. Proto has three timeouts overall; Codex has none.

Proto leads by 34 passing attempts, or **6.06 percentage points**. The recorded task-clustered bootstrap, which keeps each task's three repetitions together, gives a descriptive 95% interval of +1.25 to +11.05 points (20,000 resamples; seed 20260916). Related task families and development exposure limit interpretation beyond this workload.

The aggregate exceeds 90%, but that is not the same as 90% in every repetition. Proto scores 171, 167, and 169 of 187; its second repetition is 89.3%. The results support an observed lead on this full desk comparison, not a claim of universal superiority or a passed per-repetition reliability threshold.

4. Category results and repeatability

4.1 Where the aggregate lead comes from

Category	Tasks	Proto + DeepSeek	Codex + Sol	Gap (pp)
Bookkeeping	36	99/108 (91.7%)	98/108 (90.7%)	+0.9
Drafting	19	49/57 (86.0%)	37/57 (64.9%)	+21.1
Extraction	26	70/78 (89.7%)	69/78 (88.5%)	+1.3
Reformatting	26	74/78 (94.9%)	78/78 (100.0%)	-5.1
Reports	30	81/90 (90.0%)	56/90 (62.2%)	+27.8
Spreadsheet	41	115/123 (93.5%)	109/123 (88.6%)	+4.9
Tooling	9	19/27 (70.4%)	26/27 (96.3%)	-25.9

Table 3. Frozen-scorer results by category. Denominators include all three repetitions. The gap is Proto minus Codex in percentage points; category sizes differ, so category percentages must not be averaged to reconstruct the overall score.

The largest contribution to the 34-pass aggregate lead comes from **reports: 81 versus 56 passes**, a gain of 25. Drafting adds 12 passes, spreadsheet work adds six, and bookkeeping and extraction each add one. Codex leads in reformatting by four passes and tooling by seven. The arithmetic is $25 + 12 + 6 + 1 + 1 - 4 - 7 = 34$. The overall lead therefore does not imply that Proto wins every category.

Reformatting and tooling are especially useful counterexamples to a universal ranking: Codex passes 78/78 reformatting attempts and 26/27 tooling attempts, compared with Proto's 74/78 and 19/27. These small category samples describe the tested tasks, not independent estimates for every possible import workflow or business tool.

4.2 Repeatability is a separate measure

Passing attempts per task	Proto + DeepSeek	Codex + Sol
3 of 3	147	141
2 of 3	29	17
1 of 3	8	16
0 of 3	3	13

Table 4. Number of tasks with zero, one, two, or three passing attempts. Each column sums to 187 tasks; weighting rows by the number of passes recovers 507 and 473 passing attempts.

Proto passes all three attempts on **147/187 tasks (78.6%)**, versus **141/187 (75.4%)** for Codex. It fails all three attempts on three tasks, versus thirteen for Codex. The all-three measure is more demanding than attempt-level accuracy: a task with two successes contributes two passes to the headline rate but is not counted as consistently completed.

At the task-and-repetition pairing level, both systems pass 431 pairs; only Proto passes 76; only Codex passes 42; both fail 12. These counts retain every pair and recover the 34-pass difference. Repetition numbers are bookkeeping identifiers, not matched random seeds or simultaneous trials. Three observations per task do not establish production reliability, and the task-clustered interval in Section 3 should not be read as a guarantee outside this suite.

5. Worked example: payment reconciliation

5.1 What the agent receives

The `payments-match-v2` task provides three files: `bank_export.csv`, `open_invoices.csv`, and `note.txt`. The owner asks the agent to match the morning's bank credits to open invoices and return three CSV deliverables. The request names the required columns; the note states the business rules. Reference outputs, planted traps, and grader code are not supplied inside the agent container.

Deliverable	Required fields	Business purpose
unpaid.csv	invoice_id, customer, amount_outstanding	Show invoices still open and their remaining balances
matches.csv	invoice_id, line_id, amount_applied	Record which bank lines settle which invoices
unapplied.csv	line_id, amount, reason	Keep unrelated or closed-invoice credits visible

Table 5. The task's three-part delivery contract. A plausible total or explanatory message cannot replace these files or their required row-level relationships.

5.2 Rules that change the correct answer

Fees are not always unpaid balances. Stripe remittances arrive net of 2.9% plus 30 cents; the note directs the agent to treat the invoice as paid in full. International wires short by up to about 2% are also treated as paid because intermediary bank charges explain the difference. A generic partial-payment rule would leave an incorrect remainder.

A reference is not always authoritative. One credit names another customer's invoice. The note explicitly gives precedence to the payer and amount. By contrast, a payment identifying an already-closed invoice must remain unapplied; it must not be reassigned merely because another open invoice has a convenient amount.

One line can settle several invoices. A named multi-invoice ACH and a customer-only wire must be distributed across the invoices they actually cover. An exported duplicate must not be applied twice. A genuine 60% partial payment remains open for its 40% remainder. Supplier refunds and interest credits belong in the unapplied output, while debits do not.

5.3 What the checks establish

Structural checks enforce output columns and exact sets of open and unapplied identifiers. Numeric comparisons use a one-cent tolerance for outstanding balances. The custom matching check compares both applied totals and contributing bank-line identities; the named duplicate, multi-invoice, fee, wrong-reference, refund, and closed-invoice cases have additional checks.

The contract is not an assertion of perfect row-level accuracy everywhere: the general paid-invoice matching check accepts at least 90% agreement, while named edge cases impose their own requirements. A task pass means all required checks pass under those declared thresholds. It is not a claim that every conceivable business property has been proved. In the reported full comparison, both systems pass this task in all three repetitions; it illustrates the evaluation contract rather than explaining the aggregate performance gap.

6. Scoring integrity and application acceptance

6.1 Original and frozen verdicts

The original runner grades outputs immediately after execution. The frozen package is applied separately to retained outputs so both systems use the same scoring definitions. The released ledger preserves both verdicts; passing artifacts are not repaired before the frozen score is recorded.

Original verdict → Frozen verdict	Proto + DeepSeek	Codex + Sol
Pass to pass	447	430
Fail to pass	56	40
Ungraded to pass	4	3
Pass to fail	0	1
Fail to fail	54	87

Table 6. Original-to-frozen verdict transitions across all 561 attempts per system. Original grader errors are labeled ungraded rather than recast as ordinary agent failures. Frozen scoring produced no grader errors.

Proto's 447 original passes remain passes; 56 original failures and four ungraded attempts pass the frozen scorer, yielding 507. Codex retains 430 of 431 original passes, gains 40 passes from original failures and three from ungraded attempts, yielding 473. The one Codex pass-to-fail transition remains included. These transitions describe evaluator differences on the same artifacts, not improvement from rerunning an agent.

6.2 The evidence chain

Each exported attempt carries a hash of its original result, the frozen receipt, the scorer manifest, and retained output artifacts. The exporter checks the original-result hash against the stored file and checks the receipt's scorer identity and verdict. The local verifier checks all files named by the immutable scorer manifest. Hash agreement binds a claim to specific evidence; it does not itself prove that the grader implements the right business rule.

The scorer strips workbook caches before native recalculation, checks readable deliverables, and applies declared structural, numeric, text, and custom checks. It includes conservative equivalences for valid alternative representations. The preserved scorer source makes these decisions inspectable. The original grader and frozen scorer remain separate entry points, so future campaigns must declare which verdict is primary.

6.3 Build-track acceptance protocol

A build sequence has four stages: initial application delivery, then three requested changes to the same workspace. At every stage, the tester verifies the handed-over URL and credentials, evaluates applicable checklist items, and rechecks previously passing requirements. `RESULT.json` carries the URL, administrator and restricted-user logins, notes, start command, and port. Logins must work; their mere presence in JSON is insufficient.

Acceptance covers record counts and values, invitations, role boundaries, direct-request authorization, live scoped dashboards, validation, audit records, persistence, and domain-specific rules. A hidden UI control is not proof of server-side authorization. A restarted local app can support inspection but does not establish that the originally supplied public URL remained available.

Report initial checklist success, new change-item success, and regressions separately, with time and cost across the sequence. A completed tester sheet must distinguish observed pass, observed failure, and untested requirements. The release provides all 20 task packs and the runner/probe infrastructure, but does not promote automated URL probes into a completed build leaderboard.

7. Resource use

Measure	Proto + DeepSeek	Codex + Sol
Input tokens	823,587,929	165,959,457
Cached input	775,672,448	150,329,088
Uncached input	47,915,481	15,630,369
Output tokens	20,910,907	4,823,643
Estimated model cost (USD)	22.06	219.13
Median task duration (s)	124.9	135.2
p90 task duration (s)	455.6	403.3
Summed task duration (s)	113,460.1	111,218.8

Table 7. Resources across all 561 attempts per system. Costs are captured-usage API-equivalent estimates under the recorded price table, not subscription invoices or reconciled provider bills. Summed task duration is not elapsed campaign time when attempts run concurrently.

Proto has lower estimated model cost and median duration, but uses more tokens, slightly more summed task time, and a worse p90. Lower cost therefore does not imply uniformly lower resource use. Cache fractions are token-weighted; uncached totals are reported explicitly.

Proto records 16,954 completed model responses; Codex records 561 task-level usage aggregates, not API requests. These counts are not directly comparable, and separate Codex reasoning-token counts are unavailable. Nine Proto request starts lack completed responses; their additional usage is unknown. First-token latency and turn-start versus intra-turn cache splits are not available.

8. Validity and limitations

Task validity. Model-assisted authoring and mechanical controls do not replace independent practitioner review or human baselines. The organization developing the suite also develops Proto. Exposure during benchmark-driven development limits claims about unseen generalization.

Scoring coverage. A shared scorer controls grading drift, not all measurement error. Its checks are not exhaustive manual acceptance. Supplemental visual and semantic findings remain separate from the primary score; desk results establish no build-track performance.

Configuration and timing. Models and sampling controls differ, and cohorts were not contemporaneous. Provider behavior, tools, skills, and runtime choices may contribute. This comparison identifies an end-to-end result, not a causal harness-only effect.

Reproduction. Source, tasks, hashes, and score records are released; credentials, traces, historical agent binaries, and raw generated artifacts are not. Hashes identify evidence but cannot reconstruct it. The scorer accepts supplied workspaces, and the runners support new, explicitly configured experiments.

9. Conclusion

Business Harness Bench evaluates usable business deliverables. On the complete desk comparison, Proto with DeepSeek V4.1 Flash scores **90.4%**, versus **84.3%** for Codex with GPT-5.6-sol under the same frozen scorer. The release separates correctness, repeatability, execution status, and cost. Application tasks extend the delivery contract to behavior under change; their acceptance results remain unclaimed.

References

1. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? arXiv:2403.07718, 2024.
<https://arxiv.org/abs/2403.07718>
2. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. arXiv:2406.14991, 2024.
<https://arxiv.org/abs/2406.14991>
3. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. arXiv:2412.14161, 2024.
<https://arxiv.org/abs/2412.14161>
4. EnterpriseClawBench: Benchmarking Agents from Real Workplace Sessions. arXiv:2606.23654, 2026.
<https://arxiv.org/abs/2606.23654>

Appendix A. Running and auditing the benchmark

A.1 Installation and execution

Use Python 3.11 or 3.12 and Docker on a dedicated Linux evaluation host. Install requirements.txt and build the agent and recalculation images with docker/build.sh. Provision a benchmark-owned model login; private agent runtimes are supplied separately. The README contains full commands for each supported adapter and both tracks.

The desk runner mounts only the workspace, isolated home, and read-only adapter into the agent container. References remain on the host. Local-process mode is for development: it does not isolate host-side answers. Network access remains enabled for model requests and permitted tooling; containers are not a guarantee against online discovery of an exposed task set.

Use a fresh result label for each run. The runner refuses to overwrite existing desk labels or attempt directories. Build runs require an explicitly allocated port and a dedicated test environment. Do not expose unreviewed generated applications or mount production credentials. Build tester sheets include generated logins and must remain private.

A.2 Scoring and verification

```
python -m unittest discover -s tests -v
python bench/export_campaign.py --verify
python bench/release_manifest.py
python scoring/frozen-v7/scorer.py TASK_ID WORKSPACE
```

Set BENCH_RECALC_DOCKER_IMAGE to the native recalculation image when scoring workbooks. The last command uses the frozen scorer directly; bench/run.py records the original grader's result. Preserve both fields when publishing a new frozen-scored campaign. Never silently substitute a subset of rescored attempts into a raw aggregate.

The canonical manifest digest is:

```
b4720db00f2a55461ca70767a4baf8dd2d25aa9d301e8fe22ea13d21ccca358f
```

The package preserves its original assembly-status text to keep its fingerprint immutable; the subsequent recorded replay covered 1,341 attempts with zero verdict differences. Current export verification checks the receipt and source hashes rather than editing that historical manifest.

A.3 Release validation

All 187 desk reference-solution/untouched-workspace checks passed with native recalculation; all 20 build-task packs passed structural and seed validation. The strict desk validator reported 28 supplied-fixture byte differences from regenerated outputs despite deterministic repeated regeneration. The supplied inputs remain authoritative and unchanged. Affected tasks are listed in docs/validation.md; byte differences are not assumed semantically harmless.

Appendix B. Reproduction and metric definitions

B.1 Launching a new complete desk run

The following sequence installs the benchmark and launches the Codex desk matrix after a benchmark-owned login has been provisioned as described in README.md. It makes paid model calls. Use a dedicated Linux host, a unique label, and the exact intended adapter configuration; the example is a new experiment, not a reconstruction of a private historical agent environment.

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
bash docker/build.sh
export BENCH_RECASC_DOCKER_IMAGE=bench-recalc:release
python bench/run.py --docker business-bench:release \
  --harness codex-sol --tasks all --runs 3 \
  --parallel 2 --label reproduction-codex
```

Start with a single-task smoke run under a different label before the full matrix. A different model, provider route, skill inventory, or runtime revision defines a different cell and must be named accordingly. Proto adapters require a separately supplied runtime and isolated model configuration; platform-backed build runs additionally require dedicated test-organization access.

B.2 Minimum reproducibility record

Record	Required content
Workload	Task identifiers, task/input hashes, repetition mapping
Agent environment	Runtime revision, image digest, model route, tools and skills
Sampling and budget	Reasoning setting, temperature where controllable, timeout, CPU and memory
Scoring	Frozen package digest, original and primary verdicts, grader errors
Evidence	Original result hash, output hashes, private artifact retention location
Accounting	Captured usage, price assumptions, durations, missing observations

Table 8. Minimum record for a new campaign. Missing fields must be disclosed rather than inferred from a successful result. These are prospective requirements, not a claim that every historical environment detail is reconstructible.

B.3 Metric definitions

Attempt pass rate is the number of frozen passing attempts divided by all attempts in the complete matrix. **All-three success** is the fraction of task identifiers with three passing attempts. The latter gives one task one vote; the former includes each scheduled attempt.

Estimated model cost sums uncached input, cached input, and output tokens multiplied by their respective per-token prices. Uncached input equals total input minus cached input. The aggregate cache fraction is total cached input divided by total input, not the mean of per-request percentages. Missing usage remains unknown.

Duration is reported at the attempt level. The median describes the center of the observed durations; p90 uses the nearest-rank observation. Summing attempt durations measures aggregate occupied task time and can exceed campaign wall time under concurrency. A positive accuracy gap is stated in percentage points, not as a relative percentage reduction in error.